

Taller: Patrón de diseño Builder

Estudiante: Anggel Felipe Leal Rivera

Código: 20241020021

Institución: Universidad Distrital Francisco José de Caldas

Materia: Modelos de Programación

Parte 1: Preguntas conceptuales

1. ¿Qué problema resuelve el patrón Builder? Mencione al menos dos desventajas del "constructor telescópico".

El patrón Builder resuelve el problema de la inicialización de objetos complejos que requieren múltiples parámetros, permitiendo construirlos paso a paso.

Desventajas del constructor telescópico:

- **Baja legibilidad y facilidad de uso:** Al tener múltiples constructores encadenados, es fácil confundir el orden de los parámetros, especialmente si comparten el mismo tipo de dato, como por ejemplo varios boolean seguidos.
- **Inflexibilidad:** Obliga a instanciar la clase pasando valores nulos o falsos para los parámetros opcionales que no se desean configurar en ese momento.

2. Nombre los 4 roles clásicos del patrón y explica en una frase qué hace cada uno.

- **Product:** Es la clase compleja resultante que se está construyendo.
- **Builder:** Define la interfaz genérica o los pasos abstractos necesarios para ensamblar las diferentes partes del producto.
- **ConcreteBuilder:** Implementa los métodos de la interfaz Builder para construir un producto específico, guardando el estado intermedio de la construcción.
- **Director:** Define y controla el orden lógico en el que se deben llamar los pasos del Builder para generar una configuración u objeto particular.

3. ¿Por qué se suele omitir el rol Director?

Se suele omitir, especialmente cuando se aplica la variante de "encadenamiento de métodos", porque la lógica de instanciación se simplifica. El propio cliente asume la responsabilidad de encadenar los métodos según sus necesidades, haciendo innecesaria una clase adicional para dirigir la construcción.

4. ¿Cuál es la diferencia entre la construcción y la representación final del objeto en el patrón Builder?

La construcción es un proceso mutable, temporal y progresivo manejado por el Builder, donde se van definiendo los atributos. La representación final es el objeto ya ensamblado y devuelto, es decir el Product, el cual queda listo para ser usado y por lo general es inmutable.

Parte 2: Refactorización

Refactorización de la clase Pizza aplicando el patrón Builder mediante encadenamiento de métodos (Builder estático interno):

```
public class Pizza {
    private final String masa;
    private final String tamano;
    private final boolean queso;
    private final boolean pepperoni;
    private final boolean champinones;
    private final boolean pina;
    private final String salsa;

    private Pizza(Builder builder) {
        this.masa = builder.masa;
        this.tamano = builder.tamano;
        this.queso = builder.queso;
        this.pepperoni = builder.pepperoni;
        this.champinones = builder.champinones;
        this.pina = builder.pina;
        this.salsa = builder.salsa;
    }

    public static class Builder {
        private String masa;
        private String tamano;
        private boolean queso;
        private boolean pepperoni;
        private boolean champinones;
        private boolean pina;
        private String salsa;

        public Builder masa(String masa) { this.masa = masa; return this; }
        public Builder tamano(String tamano) { this.tamano = tamano; return this; }
        public Builder queso(boolean queso) { this.queso = queso; return this; }
        public Builder pepperoni(boolean pepperoni) { this.pepperoni = pepperoni;
return this; }
        public Builder champinones(boolean champinones) { this.champinones =
```

```

champinones; return this; }
    public Builder pina(boolean pina) { this.pina = pina; return this; }
    public Builder salsa(String salsa) { this.salsa = salsa; return this; }

    public Pizza build() {
        return new Pizza(this);
    }
}
}

```

Parte 3: Identificar los roles

- **a)** interface CarroBuilder { ... } -> **Builder**
- **b)** class CarroDeportivoBuilder implements CarroBuilder { ... } -> **ConcreteBuilder**
- **c)** class CarroDirector { ... } -> **Director**
- **d)** class Carro { ... } -> **Product**

Parte 4: Encontrar el error

El problema: La clase instancia el objeto al momento de crear el builder, específicamente en la línea `private Computador computador = new Computador();`. Si se reutiliza el builder para crear dos computadores distintos llamando a `build()` más de una vez, ambos llamados retornarán la misma referencia en memoria. Modificar los atributos para el segundo computador sobrescribirá los del primero.

Cómo arreglarlo: El método `build()` debe reiniciar la instancia interna de `Computador` después de retornar el producto actual, permitiendo que el siguiente ensamblaje empiece desde cero:

```

public class ComputadorBuilder {
    private Computador computador = new Computador();

    public ComputadorBuilder procesador(String p) {
        computador.procesador = p;
        return this;
    }

    public Computador build() {
        Computador resultado = this.computador;
        this.computador = new Computador(); // Se reinicia para el próximo uso
        return resultado;
    }
}

```

```
}  
}
```

Parte 5: Para argumentar

Casos donde ES favorable aplicar el patrón Builder:

- **Construcción de objetos inmutables pesados:** Cuando se requiere que un objeto de configuración, como un manejador de conexión a base de datos HTTP, reciba muchos atributos que son fijos durante todo su ciclo de vida. El builder asegura que se pasen de manera clara antes de ser inmutables.
- **Variantes del mismo proceso de construcción:** Cuando el mismo flujo de datos puede crear distintos tipos de productos. Ejemplo: Un sistema que procesa datos financieros y usa un PDFReportBuilder o un ExcelReportBuilder dirigidos por el mismo Director.

Casos donde NO es favorable aplicar el patrón Builder:

- **Objetos simples o DTOs:** En clases pequeñas con pocos parámetros donde un constructor estándar es suficiente, como una clase Punto con coordenadas X y Y. Aplicarlo generaría un exceso de código o sobreingeniería.
- **Entornos de alta mutabilidad externa:** Si el objeto cambia constantemente su estructura y atributos luego de ser creado, es decir donde se usan directamente los métodos set permanentemente, el patrón Builder pierde propósito, ya que su fortaleza principal radica en el proceso de instanciación controlada y consolidada.